In [ ]:
```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets pr
# You can also write temporary files to /kaggle/temp/, but they won't be saved outs
```

# Youtube Video Link : https://youtu.be/Asm2_qgBK48

In [ ]:
```
import warnings
warnings.filterwarnings('ignore')
```

In [ ]:
```
import os

for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

In [ ]:
```
# The previous code failed because the file path was incorrect.
# Based on the output of the first cell, the correct path is likely:
# /kaggle/input/imdb-data/IMDB-Movie-Data.csv

data = pd.read_csv('IMDB-Movie-Data.csv')
```

# 1. Display Top 10 Rows of The Dataset

In [ ]:
```
data.head(10)
```

Out[ ]:

| | Rank | Title | Genre | Description | Director | Actor |
|---|---|---|---|---|---|---|
| **0** | 1 | Guardians of the Galaxy | Action,Adventure,Sci-Fi | A group of intergalactic criminals are forced ... | James Gunn | Chris Pratt, Vi Diesel, Bradle Cooper, Zoe S. |
| **1** | 2 | Prometheus | Adventure,Mystery,Sci-Fi | Following clues to the origin of mankind, a te... | Ridley Scott | Noomi Rapace Logan Marshall Green, Michael Fa. |
| **2** | 3 | Split | Horror,Thriller | Three girls are kidnapped by a man with a diag... | M. Night Shyamalan | James McAvoy Anya Taylor-Joy Haley Lu Richar. |
| **3** | 4 | Sing | Animation,Comedy,Family | In a city of humanoid animals, a hustling thea... | Christophe Lourdelet | Matthev McConaughey,Rees Witherspoon, Setl Ma. |
| **4** | 5 | Suicide Squad | Action,Adventure,Fantasy | A secret government agency recruits some of th... | David Ayer | Will Smith, Jare Leto, Margo Robbie, Viola D. |
| **5** | 6 | The Great Wall | Action,Adventure,Fantasy | European mercenaries searching for black powde... | Yimou Zhang | Matt Damon, Tia Jing, Willem Dafoe Andy La |
| **6** | 7 | La La Land | Comedy,Drama,Music | A jazz pianist falls for an aspiring actress i... | Damien Chazelle | Ryan Gosling, Emm Stone, Rosemari DeWitt, J.. |
| **7** | 8 | Mindhorn | Comedy | A has-been actor best known for playing the ti... | Sean Foley | Essie Davis, Andre Riseborough, Julia Barrat. |

| | Rank | Title | Genre | Description | Director | Actor |
|---|---|---|---|---|---|---|
| **8** | 9 | The Lost City of Z | Action,Adventure,Biography | A true-life drama, centering on British explor... | James Gray | Charlie Hunnam Robert Pattinson Sienna Mille. |
| **9** | 10 | Passengers | Adventure,Drama,Romance | A spacecraft traveling to a distant colony pla... | Morten Tyldum | Jennifer Lawrence Chris Pratt, Michae Sheen,. |

# 2. Check Last 10 Rows of The Dataset

In [ ]:
```
data.tail(10)
```

Out[ ]:

| | Rank | Title | Genre | Description | Director | Actors |
|---|---|---|---|---|---|---|
| **990** | 991 | Underworld: Rise of the Lycans | Action,Adventure,Fantasy | An origins story centered on the centuries-old... | Patrick Tatopoulos | Rhona Mitra, Michael Sheen, Bill Nighy, Steven... |
| **991** | 992 | Taare Zameen Par | Drama,Family,Music | An eight-year-old boy is thought to be a lazy ... | Aamir Khan | Darsheel Safary, Aamir Khan, Tanay Chheda, Sac... |
| **992** | 993 | Take Me Home Tonight | Comedy,Drama,Romance | Four years after graduation, an awkward high s... | Michael Dowse | Topher Grace, Anna Faris, Dan Fogler, Teresa P... |
| **993** | 994 | Resident Evil: Afterlife | Action,Adventure,Horror | While still out to destroy the evil Umbrella C... | Paul W.S. Anderson | Milla Jovovich, Ali Larter, Wentworth Miller,K... |
| **994** | 995 | Project X | Comedy | 3 high school seniors throw a birthday party t... | Nima Nourizadeh | Thomas Mann, Oliver Cooper, Jonathan Daniel Br... |
| **995** | 996 | Secret in Their Eyes | Crime,Drama,Mystery | A tight-knit team of rising investigators, alo... | Billy Ray | Chiwetel Ejiofor, Nicole Kidman, Julia Roberts... |
| **996** | 997 | Hostel: Part II | Horror | Three American college students studying abroa... | Eli Roth | Lauren German, Heather Matarazzo, Bijou Philli... |
| **997** | 998 | Step Up 2: The Streets | Drama,Music,Romance | Romantic sparks occur between two dance studen... | Jon M. Chu | Robert Hoffman, Briana Evigan, Cassie Ventura,... |

| | Rank | Title | Genre | Description | Director | Actors |
|---|---|---|---|---|---|---|
| **998** | 999 | Search Party | Adventure,Comedy | A pair of friends embark on a mission to reuni... | Scot Armstrong | Adam Pally, T.J. Miller, Thomas Middleditch,Sh... |
| **999** | 1000 | Nine Lives | Comedy,Family,Fantasy | A stuffy businessman finds himself trapped ins... | Barry Sonnenfeld | Kevin Spacey, Jennifer Garner, Robbie Amell,Ch... |

# 3. Find Shape of Our Dataset (Number of Rows And Number of Columns)

In [ ]:
```
data.shape
```

Out[ ]:
```
(1000, 12)
```

In [ ]:
```
print('Number of Rows',data.shape[0])
print('Number of Columns',data.shape[1])
```

```
Number of Rows 1000
Number of Columns 12
```

# 4. Getting Information About Our Dataset Like Total Number Rows, Total Number of Columns, Datatypes of Each Column And Memory Requirement

In [ ]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Rank                1000 non-null   int64
 1   Title               1000 non-null   object
 2   Genre               1000 non-null   object
 3   Description         1000 non-null   object
 4   Director            1000 non-null   object
 5   Actors              1000 non-null   object
 6   Year                1000 non-null   int64
 7   Runtime (Minutes)   1000 non-null   int64
 8   Rating              1000 non-null   float64
 9   Votes               1000 non-null   int64
 10  Revenue (Millions)  872 non-null    float64
 11  Metascore           936 non-null    float64
dtypes: float64(3), int64(4), object(5)
memory usage: 93.9+ KB
```

# 5. Check Null Values In The Dataset

In [ ]:
```
data.isnull().sum()
```

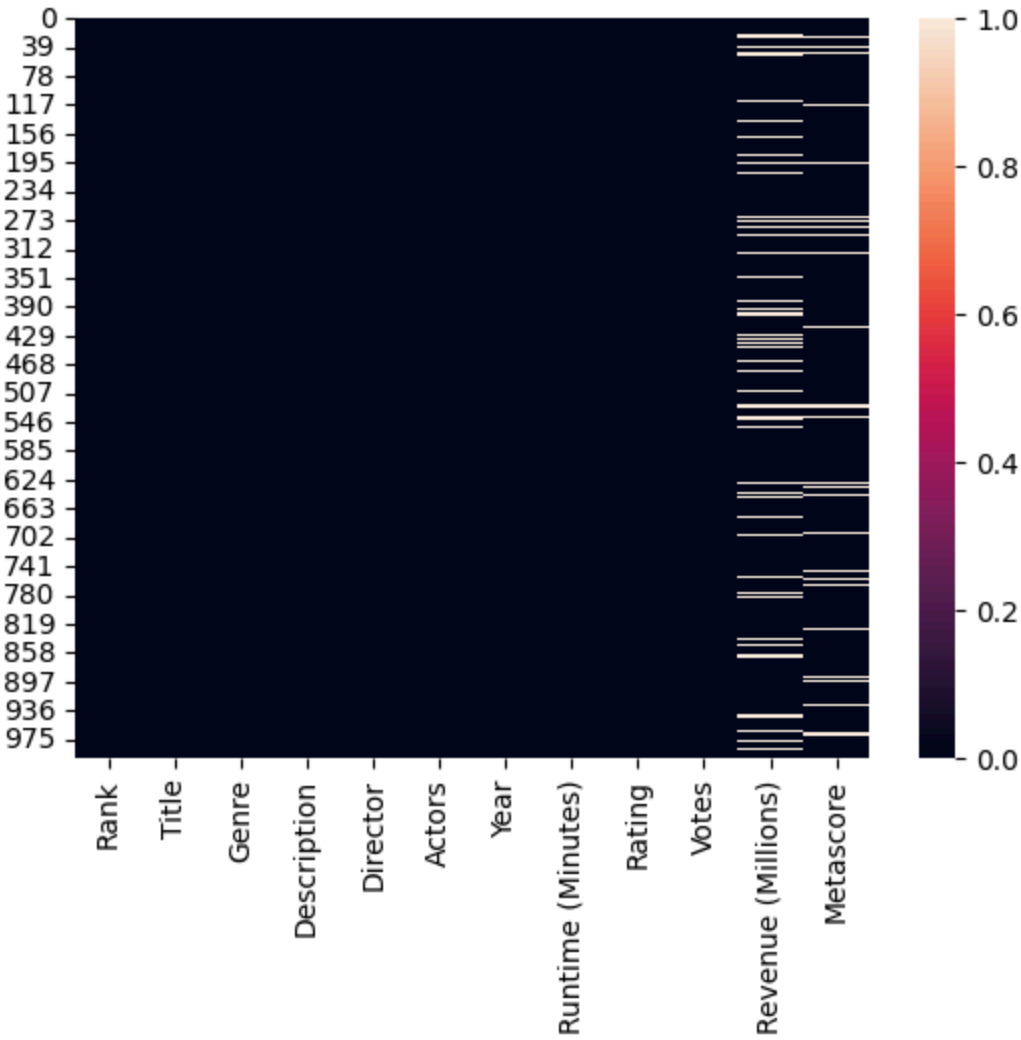
Out[ ]:

| | 0 |
|---|---|
| **Rank** | 0 |
| **Title** | 0 |
| **Genre** | 0 |
| **Description** | 0 |
| **Director** | 0 |
| **Actors** | 0 |
| **Year** | 0 |
| **Runtime (Minutes)** | 0 |
| **Rating** | 0 |
| **Votes** | 0 |
| **Revenue (Millions)** | 128 |
| **Metascore** | 64 |

**dtype:** int64

In [ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

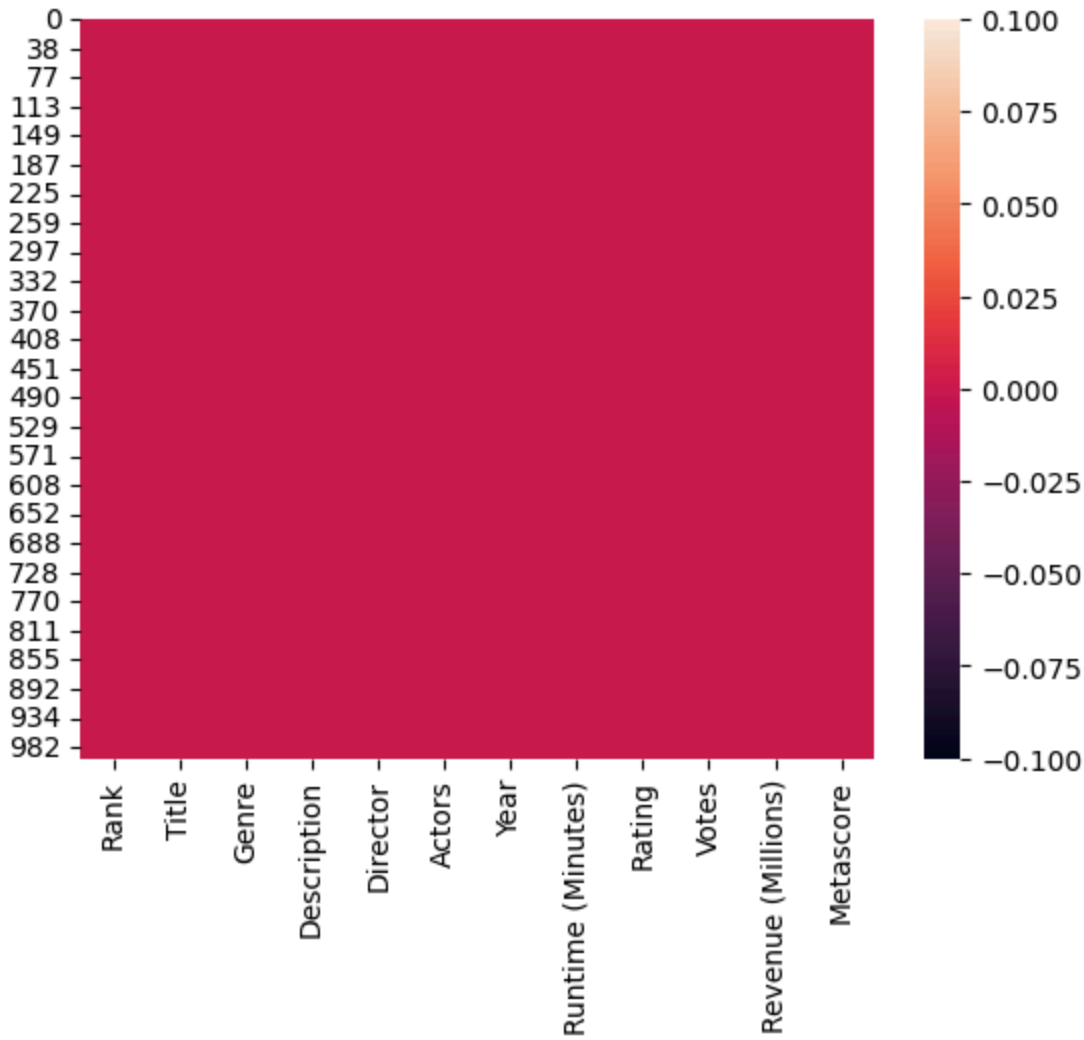
```
sns.heatmap(data.isnull())
plt.show()
```




# 6. Drop All The Missing Values

In [ ]:
```
data = data.dropna(axis=0)
```

In [ ]:
```
sns.heatmap(data.isnull())
plt.show()
```




# 7. Check For Duplicate Data

In [ ]:
```
dup_data=data.duplicated().any()
print("Are there any duplicated values in data?",dup_data)
```

```
Are there any duplicated values in data? False
```

# 8. Get Overall Statistics About The DataFrame

In [ ]:
```
data.describe()
```

Out[ ]:

| | Rank | Year | Runtime (Minutes) | Rating | Votes | Revenue (Millions) | Metasc |
|---|---|---|---|---|---|---|---|
| **count** | 838.000000 | 838.00000 | 838.000000 | 838.000000 | 8.380000e+02 | 838.000000 | 838.000 |
| **mean** | 485.247017 | 2012.50716 | 114.638425 | 6.814320 | 1.932303e+05 | 84.564558 | 59.575 |
| **std** | 286.572065 | 3.17236 | 18.470922 | 0.877754 | 1.930990e+05 | 104.520227 | 16.952 |
| **min** | 1.000000 | 2006.00000 | 66.000000 | 1.900000 | 1.780000e+02 | 0.000000 | 11.000 |
| **25%** | 238.250000 | 2010.00000 | 101.000000 | 6.300000 | 6.127650e+04 | 13.967500 | 47.000 |
| **50%** | 475.500000 | 2013.00000 | 112.000000 | 6.900000 | 1.368795e+05 | 48.150000 | 60.000 |
| **75%** | 729.750000 | 2015.00000 | 124.000000 | 7.500000 | 2.710830e+05 | 116.800000 | 72.000 |
| **max** | 1000.000000 | 2016.00000 | 187.000000 | 9.000000 | 1.791916e+06 | 936.630000 | 100.000 |

# 9. Display Title of The Movie Having Runtime >= 180 Minutes

In [ ]:
```
data[data['Runtime (Minutes)']>=180]['Title']
```

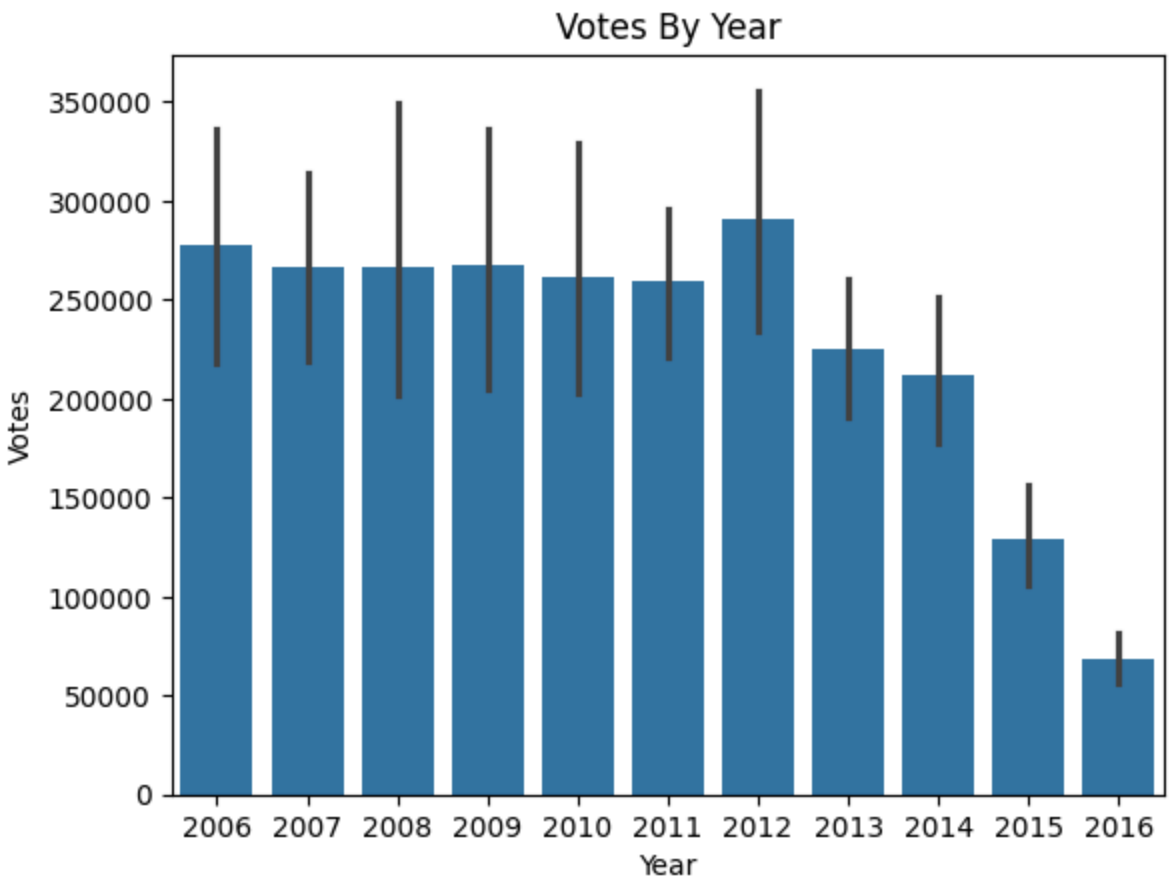
Out[ ]:

| | Title |
|---|---|
| **82** | The Wolf of Wall Street |
| **88** | The Hateful Eight |
| **311** | La vie d'Adèle |

**dtype:** object

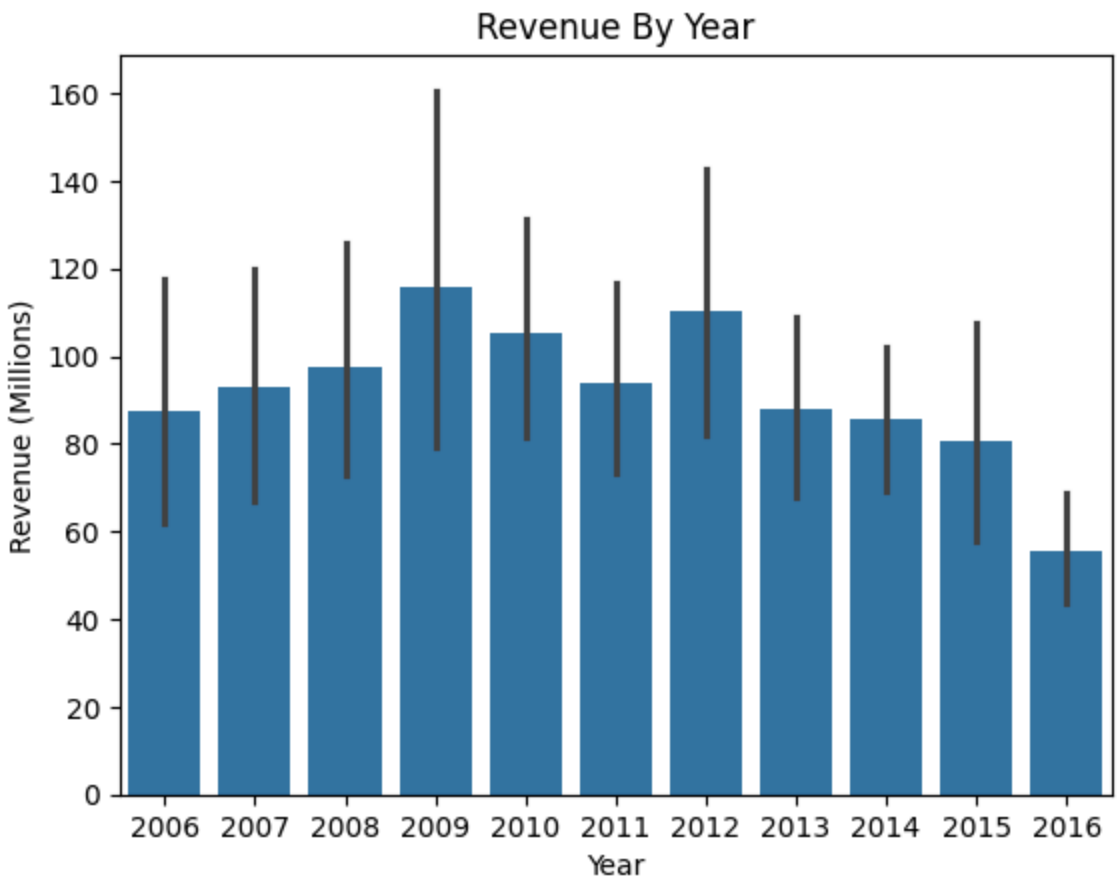
# 10. In Which Year There Was The Highest Voting?

In [ ]:
```
sns.barplot(x='Year',y='Votes',data=data)
plt.title("Votes By Year")
plt.show()
```




# 11. In Which Year There Was The Highest Revenue?

In [ ]:
```
sns.barplot(x='Year',y='Revenue (Millions)',data=data)
plt.title("Revenue By Year")
plt.show()
```




# 12. Find The Average Rating For Each Director

In [ ]:
```
data.groupby('Director')['Rating'].mean().sort_values(ascending=False)
```

Out[ ]:

| Director | Rating |
|---|---|
| **Christopher Nolan** | 8.68 |
| **Makoto Shinkai** | 8.60 |
| **Olivier Nakache** | 8.60 |
| **Aamir Khan** | 8.50 |
| **Florian Henckel von Donnersmarck** | 8.50 |
| **...** | ... |
| **Sam Taylor-Johnson** | 4.10 |
| **Joey Curtis** | 4.00 |
| **George Nolfi** | 3.90 |
| **James Wong** | 2.70 |
| **Jason Friedberg** | 1.90 |

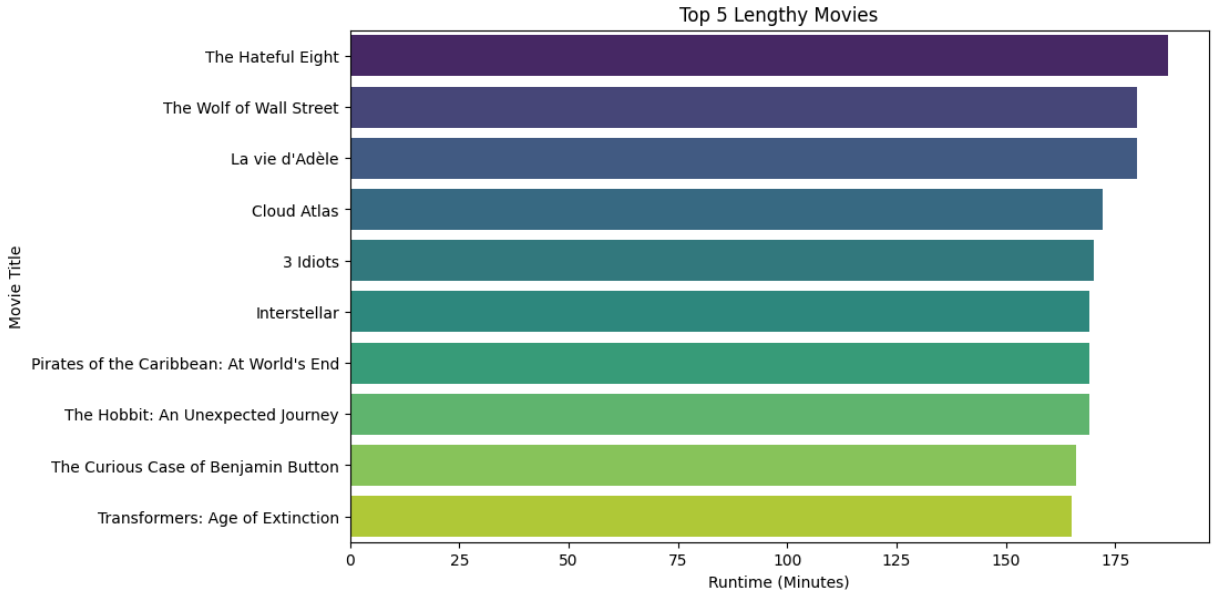
524 rows × 1 columns

**dtype:** float64

# 13. Display Top 10 Lengthy Movies Title

In [ ]:
```
le =data.nlargest(10,'Runtime (Minutes)')[['Title','Runtime (Minutes)']]. \
set_index('Title')
```

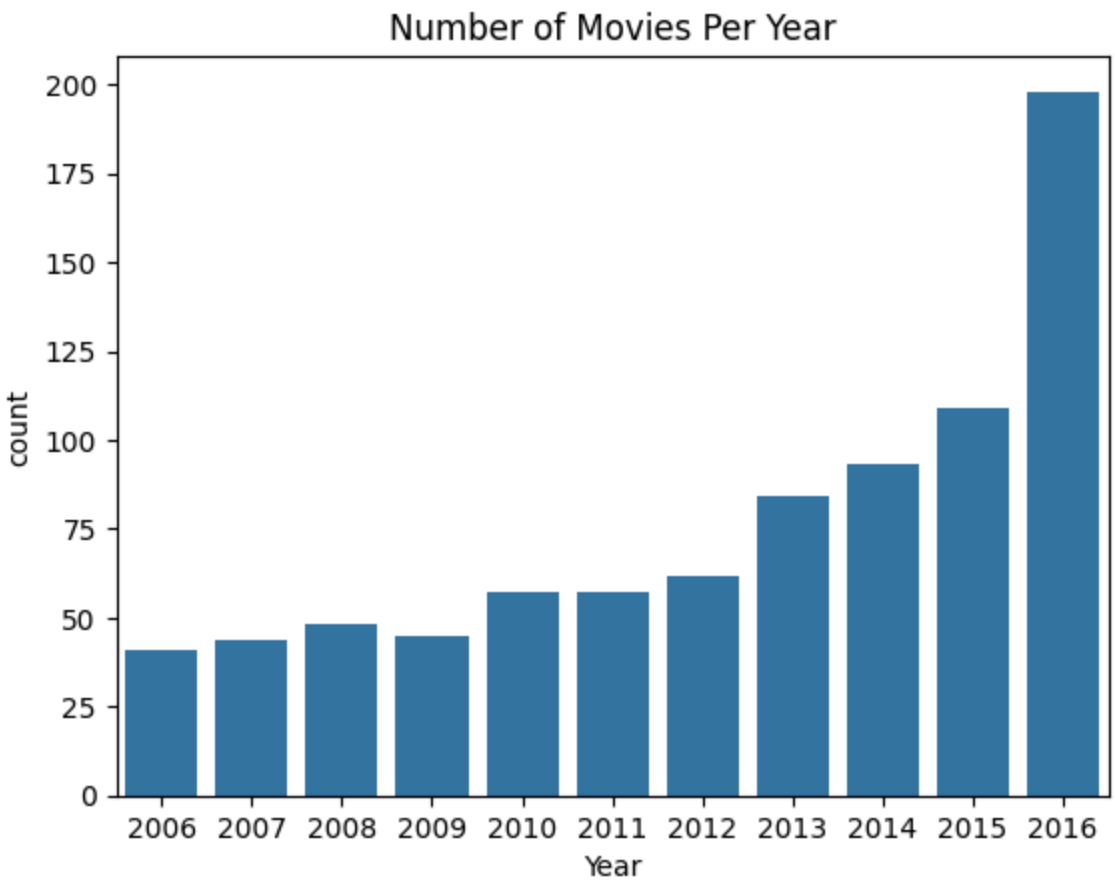
In [ ]:
```
le = le.reset_index()

plt.figure(figsize=(10,6))
sns.barplot(x='Runtime (Minutes)', y='Title', data=le, palette='viridis')
plt.title('Top 5 Lengthy Movies')
plt.xlabel('Runtime (Minutes)')
plt.ylabel('Movie Title')
plt.show()
```




# 14. Display Number of Movies Per Year

In [ ]:
```
sns.countplot(x='Year',data=data)
plt.title("Number of Movies Per Year")
```

Out[ ]: Text(0.5, 1.0, 'Number of Movies Per Year')



# 15. Find Most Popular Movie Title (Higest Revenue)

In [ ]:
```
data.columns
```

Out[ ]:
```
Index(['Rank', 'Title', 'Genre', 'Description', 'Director', 'Actors', 'Year',
       'Runtime (Minutes)', 'Rating', 'Votes', 'Revenue (Millions)',
       'Metascore'],
      dtype='object')
```

In [ ]:
```
data[data['Revenue (Millions)'].max() == data['Revenue (Millions)']]['Title']
```

Out[ ]:

| | Title |
|---|---|
| **50** | Star Wars: Episode VII - The Force Awakens |

**dtype:** object

# 16. Display Top 10 Highest Rated Movie Titles And its Directors

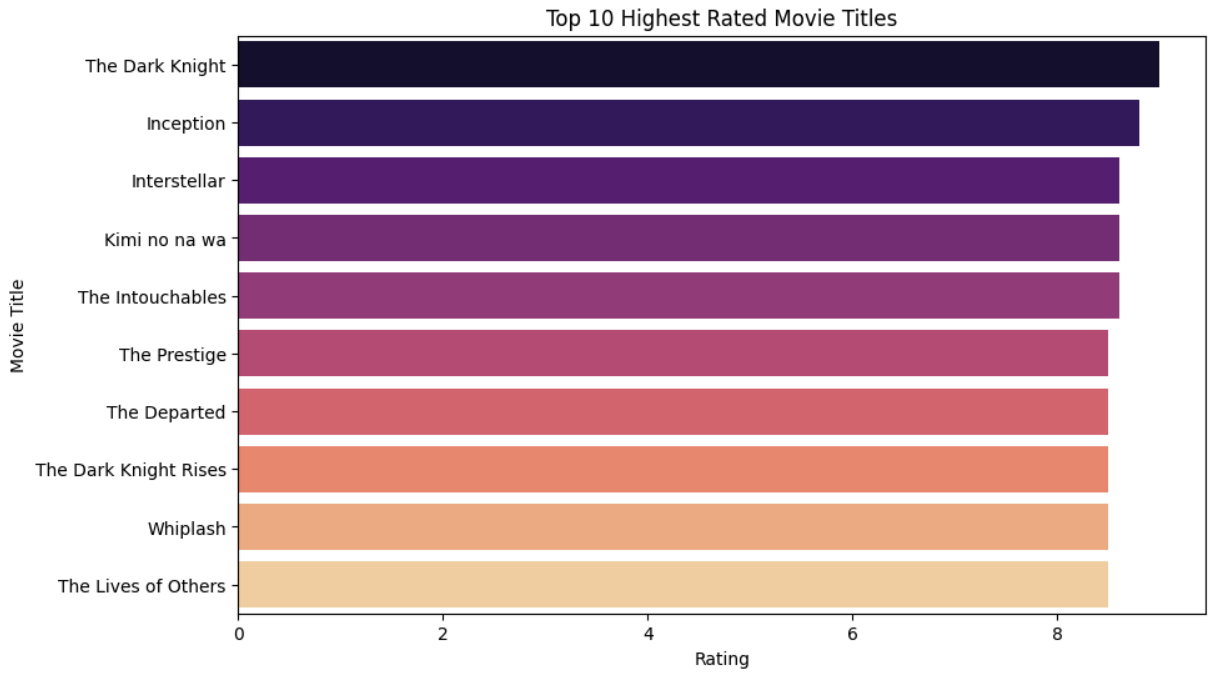
In [ ]:
```
top_10=data.nlargest(10,'Rating')[['Title','Rating','Director']].set_index('Title')
```

In [ ]:
```
top_10
```

Out[ ]:

| Title | Rating | Director |
|---|---|---|
| **The Dark Knight** | 9.0 | Christopher Nolan |
| **Inception** | 8.8 | Christopher Nolan |
| **Interstellar** | 8.6 | Christopher Nolan |
| **Kimi no na wa** | 8.6 | Makoto Shinkai |
| **The Intouchables** | 8.6 | Olivier Nakache |
| **The Prestige** | 8.5 | Christopher Nolan |
| **The Departed** | 8.5 | Martin Scorsese |
| **The Dark Knight Rises** | 8.5 | Christopher Nolan |
| **Whiplash** | 8.5 | Damien Chazelle |
| **The Lives of Others** | 8.5 | Florian Henckel von Donnersmarck |

In [ ]:
```
top_10 = top_10.reset_index()

plt.figure(figsize=(10, 6))
sns.barplot(x='Rating', y='Title', data=top_10, palette='magma')
plt.title("Top 10 Highest Rated Movie Titles")
plt.xlabel("Rating")
plt.ylabel("Movie Title")
plt.show()
```




# 17. Display Top 10 Highest Revenue Movie Titles

In [ ]:
```
data.columns
```

Out[ ]:
```
Index(['Rank', 'Title', 'Genre', 'Description', 'Director', 'Actors', 'Year',
       'Runtime (Minutes)', 'Rating', 'Votes', 'Revenue (Millions)',
       'Metascore'],
      dtype='object')
```

In [ ]:
```
data.sort_values(by='Revenue (Millions)',ascending=False).head(10)
```

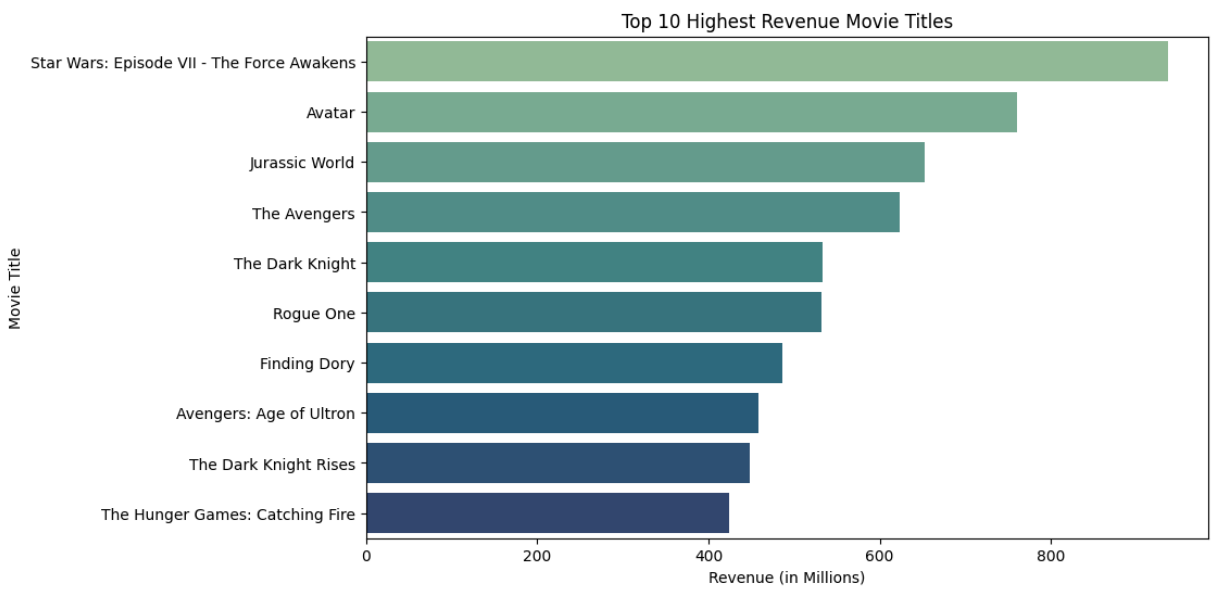
Out[ ]:

| | Rank | Title | Genre | Description | Director | Actors |
|---|---|---|---|---|---|---|
| **50** | 51 | Star Wars: Episode VII - The Force Awakens | Action,Adventure,Fantasy | Three decades after the defeat of the Galactic... | J.J. Abrams | Daisy Ridley, John Boyega, Oscar Isaac, Domhna... |
| **87** | 88 | Avatar | Action,Adventure,Fantasy | A paraplegic marine dispatched to the moon Pan... | James Cameron | Sam Worthington, Zoe Saldana, Sigourney Weaver... |
| **85** | 86 | Jurassic World | Action,Adventure,Sci-Fi | A new theme park, built on the original site o... | Colin Trevorrow | Chris Pratt, Bryce Dallas Howard, Ty Simpkins,... |
| **76** | 77 | The Avengers | Action,Sci-Fi | Earth's mightiest heroes must come together an... | Joss Whedon | Robert Downey Jr., Chris Evans, Scarlett Johan... |
| **54** | 55 | The Dark Knight | Action,Crime,Drama | When the menace known as the Joker wreaks havo... | Christopher Nolan | Christian Bale, Heath Ledger, Aaron Eckhart,Mi... |
| **12** | 13 | Rogue One | Action,Adventure,Sci-Fi | The Rebel Alliance makes a risky move to steal... | Gareth Edwards | Felicity Jones, Diego Luna, Alan Tudyk, Donnie... |
| **119** | 120 | Finding Dory | Animation,Adventure,Comedy | The friendly but forgetful blue tang fish, Dor... | Andrew Stanton | Ellen DeGeneres, Albert Brooks,Ed O'Neill, Kai... |
| **94** | 95 | Avengers: Age of Ultron | Action,Adventure,Sci-Fi | When Tony Stark and Bruce Banner try to jump-s... | Joss Whedon | Robert Downey Jr., Chris Evans, Mark Ruffalo, ... |

| | Rank | Title | Genre | Description | Director | Actors |
|---|---|---|---|---|---|---|
| **124** | 125 | The Dark Knight Rises | Action,Thriller | Eight years after the Joker's reign of anarchy... | Christopher Nolan | Christian Bale, Tom Hardy, Anne Hathaway,Gary ... |
| **578** | 579 | The Hunger Games: Catching Fire | Action,Adventure,Mystery | Katniss Everdeen and Peeta Mellark become targ... | Francis Lawrence | Jennifer Lawrence, Josh Hutcherson, Liam Hemsw... |

In [ ]:
```
top_10 = data.nlargest(10,'Revenue (Millions)')[['Title','Director','Revenue (Milli
```

In [ ]:
```
top_10 = top_10.reset_index()

plt.figure(figsize=(10, 6))
sns.barplot(x='Revenue (Millions)', y='Title', data=top_10, palette='crest')
plt.title("Top 10 Highest Revenue Movie Titles")
plt.xlabel("Revenue (in Millions)")
plt.ylabel("Movie Title")
plt.show()
```




# 18. Find Average Rating of Movies Year-wise

In [ ]:
```
data.columns
```

Out[ ]:
```
Index(['Rank', 'Title', 'Genre', 'Description', 'Director', 'Actors', 'Year',
       'Runtime (Minutes)', 'Rating', 'Votes', 'Revenue (Millions)',
       'Metascore'],
      dtype='object')
```

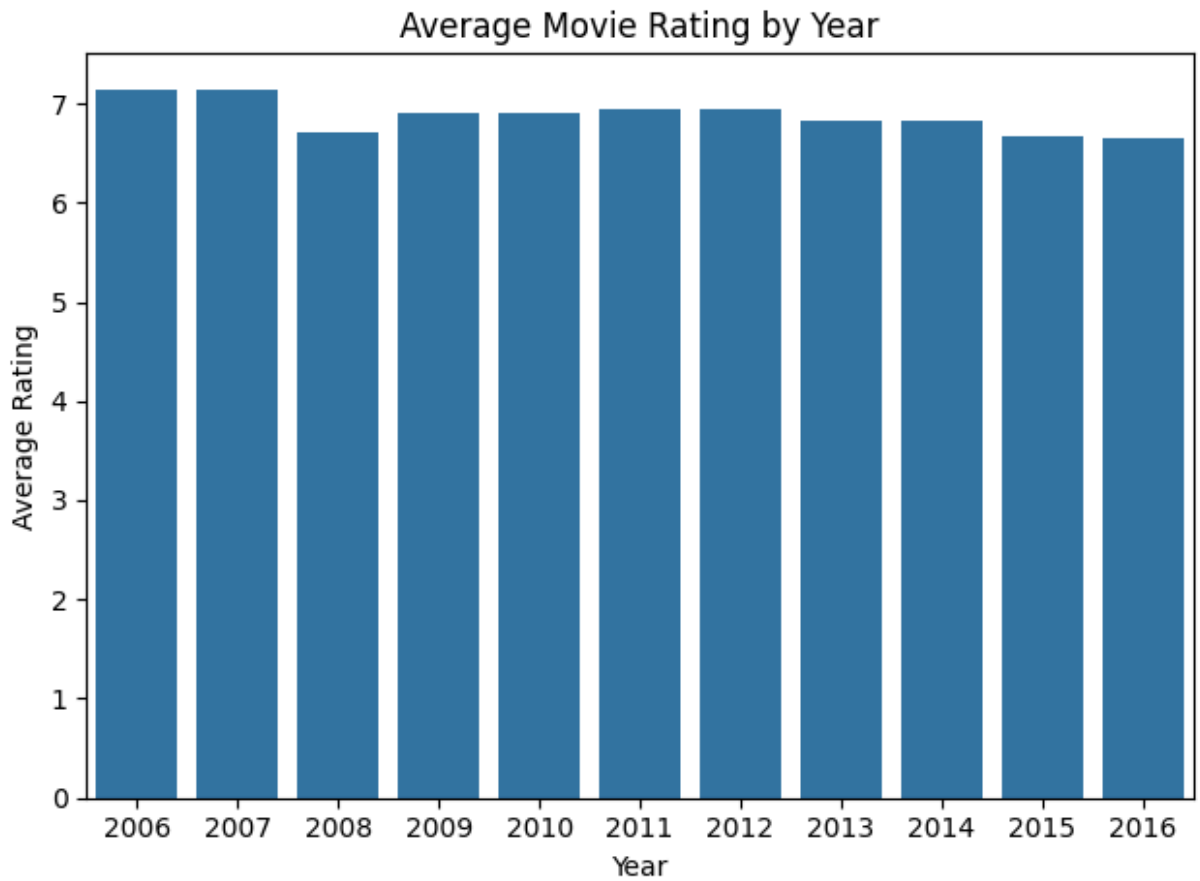
In [ ]:
```
data1 = data.groupby('Year').mean(numeric_only=True)[['Rating']].sort_values(by='Ra
```

In [ ]:
```
data1
```

Out[ ]:

| | Rating |
|---|---|
| **Year** | |
| **2006** | 7.143902 |
| **2007** | 7.140909 |
| **2011** | 6.945614 |
| **2012** | 6.933871 |
| **2009** | 6.911111 |
| **2010** | 6.894737 |
| **2013** | 6.832143 |
| **2014** | 6.822581 |
| **2008** | 6.708333 |
| **2015** | 6.674312 |
| **2016** | 6.644444 |

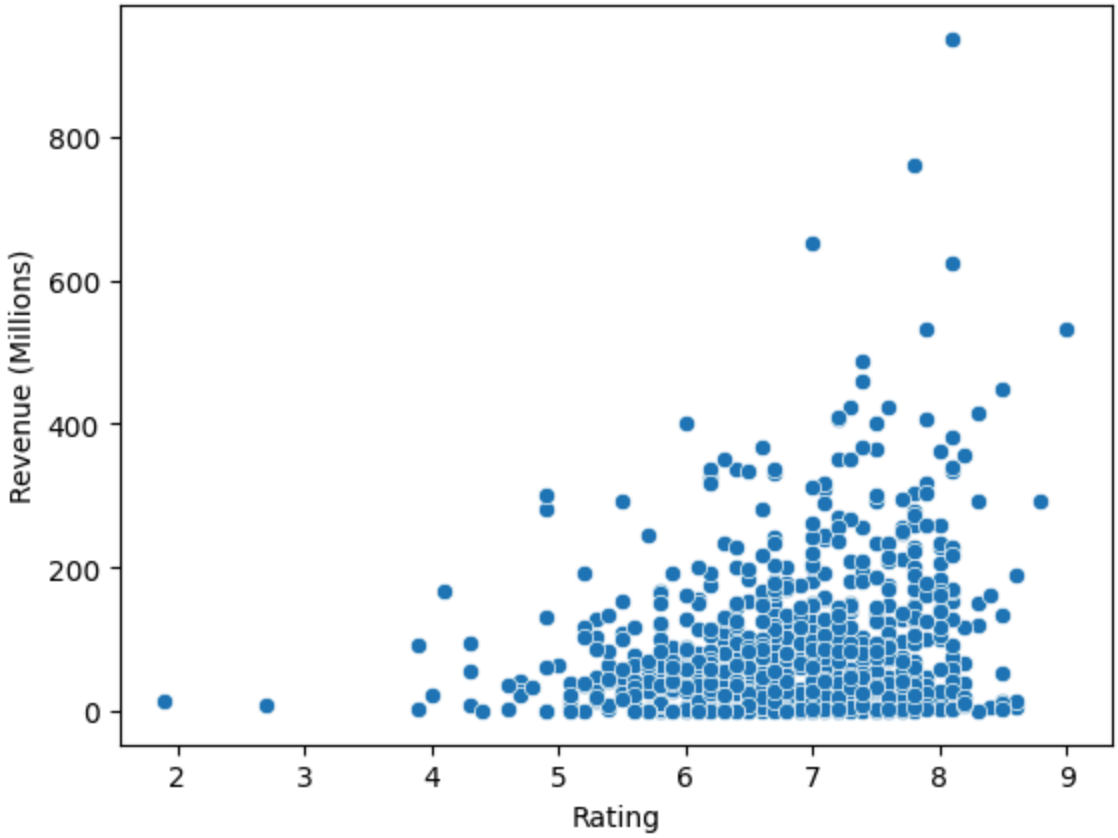
In [ ]:
```
sns.barplot(x=data1.index, y=data1['Rating'])
plt.xlabel("Year")
plt.ylabel("Average Rating")
plt.title("Average Movie Rating by Year")
plt.tight_layout()
plt.show()
```




# 19. Does Rating Affect The Revenue?

In [ ]:
```
sns.scatterplot(x='Rating',y='Revenue (Millions)',data=data)
```

Out[ ]:
```
<Axes: xlabel='Rating', ylabel='Revenue (Millions)'>
```




# Answer : Yes

# 20. Classify Movies Based on Ratings [Good,Better and Best]

In [ ]:
```
data.columns
```

Out[ ]:
```
Index(['Rank', 'Title', 'Genre', 'Description', 'Director', 'Actors', 'Year',
       'Runtime (Minutes)', 'Rating', 'Votes', 'Revenue (Millions)',
       'Metascore'],
      dtype='object')
```

In [ ]:
```
def rating(rating):
    if rating>=7.0:
        return 'Excellent'
    elif rating>=6.0:
        return 'Good'
    else:
        return 'Average'
```

In [ ]:
```
data['rating_cat']=data['Rating'].apply(rating)
```

In [ ]:
```
data.head(1)
```

Out[ ]:

| | Rank | Title | Genre | Description | Director | Actors | Year | Runtime (Minutes) | I |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | Guardians of the Galaxy | Action,Adventure,Sci-Fi | A group of intergalactic criminals are forced ... | James Gunn | Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S... | 2014 | 121 | |

# 21. Count Number of Action Movies

In [ ]:
```
list1=[]
for value in data['Genre']:
    list1.append(value.split(','))
```

In [ ]:
```
data['temp']=list1
```

In [ ]:
```
genre=input("Enter Genre you want to count : ").title()
count=0
for value in data['temp']:
    if genre in value:
        count=count+1
print("Total Count is",count)
```

```
Enter Genre you want to count : comedy
Total Count is 250
```

# OR

In [ ]:
```
len(data[data['Genre'].str.contains('action',case=False)])
```

Out[ ]: 277

In [ ]:
```
#Top 10 Rated Movies in the Comedy Genre by Aksh Modgill


#First we filter movies that include 'Comedy' in the Genre
comedy_movies = data[data['Genre'].str.contains('Comedy', case=False, na=False)]

# Then we sort by Rating in descending order
top_10_comedy = comedy_movies.sort_values(by='Rating', ascending=False).head(10)

print("Top 10 Rated Comedy Movies:")
print(top_10_comedy[['Title', 'Rating']])
```

```
Top 10 Rated Comedy Movies:
                        Title  Rating
249          The Intouchables     8.6
430                   3 Idiots     8.4
499                         Up     8.3
6                   La La Land     8.3
688               Toy Story 3     8.3
241                Inside Out     8.2
765                         PK     8.2
82    The Wolf of Wall Street     8.2
74                   Zootopia     8.1
238  The Grand Budapest Hotel     8.1
```

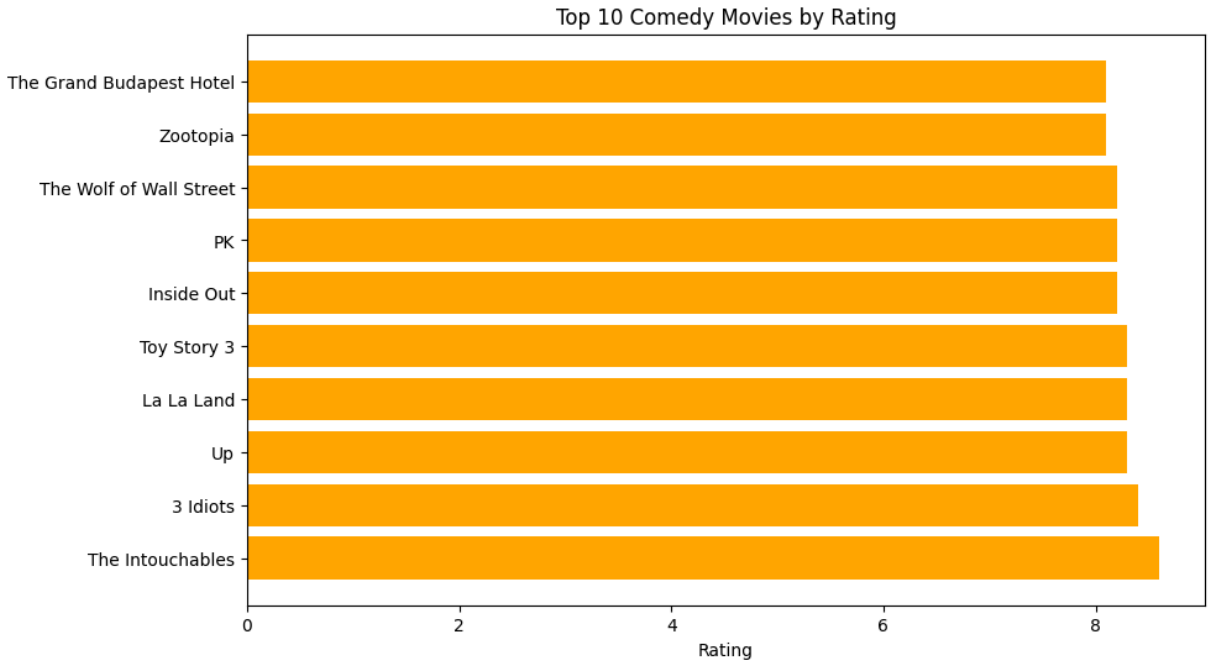
In [ ]:
```
# Top 5 Rated Movies Overall by Aksh Modgill


# First we sort or filter entire dataset by Rating
top_5_overall = data.sort_values(by='Rating', ascending=False).head(5)

print("Top 5 Rated Movies Overall:")
print(top_5_overall[['Title', 'Rating']])
```

```
Top 5 Rated Movies Overall:
                Title  Rating
54    The Dark Knight     9.0
80          Inception     8.8
36       Interstellar     8.6
96      Kimi no na wa     8.6
249  The Intouchables     8.6
```

In [ ]:
```
# Worst 5 Rated Movies Overall by Aksh Modgill

# First we sort or filter entire dataset by Rating
top_5_overall = data.sort_values(by='Rating', ascending=True).head(5)

print("Top 5 Rated Movies Overall:")
print(top_5_overall[['Title', 'Rating']])
```

```
Top 5 Rated Movies Overall:
                        Title  Rating
829            Disaster Movie     1.9
871      Dragonball Evolution     2.7
525       Birth of the Dragon     3.9
237  The Disappointments Room     3.9
616      2307: Winter's Dream     4.0
```

In [ ]:
```
# Bar plot for Top 10 Comedy Movies by Aksh Modgill
plt.figure(figsize=(10,6))
plt.barh(top_10_comedy['Title'], top_10_comedy['Rating'], color='orange')
plt.xlabel('Rating')
plt.title('Top 10 Comedy Movies by Rating')
plt.show()
```


Top 10 Comedy Movies by Rating
The Grand Budapest Hotel
Zootopia
The Wolf of Wall Street
PK
Inside Out
Toy Story 3
La La Land
Up
3 Idiots
The Intouchables
0
2
4
6
8
Rating